TRACERA

Full state and path to a valuable product

Independent product, engineering and completion review

16 October 2026 Pacific | Source ae68aea7 | Documentation delivery

Operational kernel qualified in a stopped isolated Windows/SQLite candidate. Product CVP NOT READY.

This report preserves the wider backlog while defining a complete useful core. It does not certify production, frontend acceptance, broad test coverage or an unfamiliar-user pilot.

Reading guide

1. Product state: intent, every goal cluster and present usability.
2. Engineering state: reusable foundations, unfinished behavior and evidence limits.
3. Completion path: dependencies, effort ranges, pilot, rewrite and sunset decisions.

Tracera: full product state and present usability

Review date: 2026-10-16 PDT / 2026-10-17 UTC. Independent read-only product/value review. Source pin: `ae68aea7febd772282606bf1e142d507316cfea0`, isolated checkout `/tmp/tracera-readiness-source-20261016-latest`. Repository is standalone (`fork=false`, `parent=null`, freshly resolved by engineering lane). This is completion of a project, not completion of an upstream fork.

Executive judgement

The operational kernel has a genuine isolated qualification receipt. The valuable product is not yet accepted. There is substantial reusable code; a whole-project rewrite has not been justified. The highest-value remaining work is a coherent persistent explanatory product loop, not converting unavailable routes into superficially successful responses.

Q0 is a meaningful advance: exact-current-main Windows/SQLite build, readiness, invalid/missing/empty-input controls, a real supplied-issue Story write, identical Story after process restart, and cleanup of the owned candidate. The candidate is **stopped**. This review does not assert that a user can open that candidate now, that production was changed, or that the hosted frontend works. Q0 does not prove product observations, automatic discrepancy discovery, UI navigation, PostgreSQL, install/update or unfamiliar-user value. Receipt authority: `docs/audit/TRACERA-Q0-RUNTIME-QUALIFICATION.md` on PR #1132, head `723964216ccc6b21b5488a30e501543769808e16` at review start. No new build, product test suite, runtime probe, browser observation, screenshot or recording was performed by this lane.

Intent and boundary

The controlling supplied dossier defines Tracera as a **persistent product/system model and dissatisfaction discovery** product. Its required witness is real product import, a genuine discrepancy with source evidence, bounded navigation and preserved identity through restart. Authority: external `CodeProjects/docs/docs-5/products/tracera/DOSSIER.md:5-24,55-93`; `AGENT-PROMPT.md:15-25,37-45`.

The repo README foregrounds agentic/LLM trace observability, memory and session history (`README.md:6,13-18`). Atlas is substantially ALM-oriented; the dossier explicitly says ALM must not redefine the product (`DOSSIER.md:28-30`). These are unresolved scope/intent tensions, not permission to silently discard work. Record each historical promise as retained required v1, supported later profile, superseded with evidence, or explicitly retired by the operator. A narrow CVP witnesses the product and preserves wider critical obligations (`DOSSIER.md:14`).

```
real product at revision R
  -> accepted intent + source/interface + evidence
  -> genuine, correctly scoped discrepancy
  -> bounded explanation and affected-consumer navigation
  -> revision R+1 reconciliation
  -> restart with the same identities and provenance
  -> unfamiliar-user pilot across three owned products
```

What the user can do now

There is a recipe and receipt for reproducing a narrow isolated backend exercise. A developer can build and run that profile and exercise its persisted Story path. The already-qualified candidate has been stopped; no currently served URL is certified here. No demonstrated user workflow yet imports a whole product, automatically identifies an evidence-backed discrepancy, navigates it in the real frontend, reconciles changed source and survives restart as one coherent job.

Historical receipts found 533 unique TRACERA items and zero CIVIS items; CIVIS zero does not establish successful CIVIS import. Historical navigation covered 56/59 direct routes, but only four global sidebar links were actually clicked while 40 project links were unavailable behind Project Not Found. Fourteen differently named views showed mixed records; detail-ID and redirects failed. These are historical findings, not current-main regressions: typed reads, journeys, search and snapshots have since improved. No fresh six-project completeness or hosted acceptance is inferred.

All major goal families

The table covers the 15 product/engineering goal families identified from governing intent and existing audit baselines. It is a navigable family inventory, not a claim that every historic promise is accepted or every dynamic path is exhaustively understood. Source facts are not runtime acceptance.

Goal family	Current source / receipt state	Usability and remaining finish criterion
1. Reach and identify a project	API project reads exist; SQLite projects derive only from `problems` (`sqlite_store/projects.rs:17-23,74-80`)	Item-only project discovery can disagree with existing artifacts. Canonical runtime must open the real imported product with stable IDs and permission checks.
2. Import real product/source	GitHub/Jira/AgilePlus issue/task ingest is mounted; Story/evidence/links are real separate writes (`ingest/persist.rs:24-112`)	Issue ingestion is not full product source import. Required source, accepted intent, revision anchors and evidence must import completely with explicit omissions.
3. Canonical identities and authoring	Item, product, capability, baseline and graph foundations exist; many HTTP writes explicitly unavailable	Distinguish product/repository/project/session/work item. Preserve source-qualified IDs, real validation and durable authoring through selected interfaces.
4. Correct artifact views	Persisted type-filtered specification and journey reads, pagination and missing-vs-empty distinctions exist	View X must show complete artifact X and selected details in the correct product; no mixed-table substitution. Actual frontend journey remains unaccepted.
5. Specifications overview/verification	Stored statistics preserve unknown metrics; summary/verify/run/activity unavailable (`routes/specs.rs:272-284`)	Hub totals must reconcile with complete lists; compliance/pass rate must reference actual evidence, not zeros or invented defaults.
6. Intent/source/test relationships	Product types and artifact links reusable; multiple model boundaries remain	One traceable accepted intent -> capability -> source/interface -> native result/evidence chain, with dangling and cross-product controls.
7. Discrepancy detection/triage	Freshness, coverage and contradiction functions exist	Present identity/grouping defects require independent counterexamples; durable mounted discovery and evidence-backed resolution missing/unproven.
8. Evidence/provenance/ history	Evidence records, Store methods, source anchors and baseline types exist	Immutable observations must retain revision, producer, privacy and historical status; restart/source-change cannot silently overwrite proof.
9. Search/bounded exploration	SQLite title substring search and suggestions now implemented	No semantic/symbol-index acceptance. A user must locate the known capability/finding and navigate relevant evidence with bounded effort and explicit unsupported edges.
10. QA/execution/analytic s/reporting	Definitions/read projections exist; many execution and analytics families unavailable	Test definition differs from execution/result. Ingest real native reports, execute retained workflows and derive reports with exact denominators.
11. Integrations/collaboration	Many route families return truthful 501	Every retained integration needs a real scoped roundtrip with pagination, retry, identity, conflict and privacy controls; historical breadth needs explicit disposition.
12. Agent observability/memory	README promises trace/session/memory product; components exist	Real run -> source-linked timeline -> truthful durable memory contribution to product purpose remains unaccepted; this cannot replace the governed model without a decision.
13. UI/API/MCP/CLI parity	Distinct interfaces/models and explicit demo mode exist	Required interfaces operate on the same persisted scoped identity and errors. A stack lifecycle CLI or in-memory demo is not full historical CRUD/product parity.
14. Install/security/recovery	Q0 SQLite build/readiness/Story restart qualified in disposable Windows profile	Hosted ingress, frontend, project/user authorization, PostgreSQL, supported packaging/install/update/recovery and backup fidelity remain separate gates.

Goal family	Current source / receipt state	Usability and remaining finish criterion
15. Documentation/quality/status honesty	Existing baseline indexing and audit CSVs; improved 501 honesty; useful tests	Some UI labels overclaim implemented writes. Numeric test/coverage/negative-control gates and retained-promise acceptance need executed receipts. No completion percentage is defensible.

Server-relative paths in this table are under `crates/tracera-server/src/` at the pinned SHA. Current implementation and engineering details are expanded in companion `02-ENGINEERING-STATE.md`.

The immediate semantic risks

These were independently source-inspected at the fresh pin:

- Observation ingestion explicitly holds values and idempotency keys in memory, with persistent wiring deferred (`product/ingest.rs:3-9,75-87`). A database-backed Story surviving restart does not repair this semantic durability gap.

- Freshness labels its capability ID with the product ID (`product/detectors.rs:129-130`).

- Coverage compares an observation's product ID with a capability intent ID (`product/detectors.rs:179-193`), instead of preserving distinct identities.

- Contradiction groups by product and baseline, omitting capability and observation kind, then labels product as capability (`product/detectors.rs:237-265`). Unrelated pass/fail observations can appear contradictory.

These are source-demonstrable counterexamples, not fresh failing-test receipts. Correct detector behavior must be established before a user makes decisions from findings.

What counts as unfinished, mocked or unknown

A truthful 501 is honest **unfinished functionality**, not feature completion. A legitimate test mock is not product deception. Explicitly selected MCP demo mode is not proof the configured production DB is mocked. Null/unknown metrics are correct when evidence is missing; fabricated successful empty envelopes are not. Source implementation, passing unit tests, isolated runtime proof, deployed acceptance and user value must remain separately labelled.

The engineering lane's reproducible census finds 216 unique literal route templates across 39 modules directly passed to `unavailable_routes(ROUTES)`. This excludes mixed/dynamic/other registrations and is only a lower bound, not 216 independent product capabilities. Historical 29 gap findings, 73 inventory rows, 284 API-reference methods and 652 indexed promise records have different denominators; none supplies a percentage complete.

First useful user feedback

No unfamiliar-user three-product pilot is established. Seek intent/job feedback now: which question should Tracera answer, which source/evidence is authoritative, which promises remain required? Product usability/value feedback follows a functioning CVP. A developer manually feeding an already-written finding or coaching every click cannot establish discovery or independent usability.

Pilot: begin with one owned product to debug the protocol, then three representative owned products under versioned inputs and independent oracles. Cases include an unreachable capability, stale coverage evidence, dependency change, duplicated owner and legitimate exception (`DOSSIER.md:55-81`). An unfamiliar operator must explain current behavior, find the issue, identify affected consumers and propose a bounded repair; repeat after source change and restart. Retain time/actions, interventions, errors, false positives, abstentions and failed attempts. Pre-register quantitative success thresholds against a competent manual Git/search/spec workflow before observing results. No comparative winner is claimed.

Evidence and publication boundary

This is a read-only assessment plus report files, not source repair or deployment. Source evidence is current to the pinned SHA. Q0 receipts are preserved prior executions matched to that SHA. Historical UI/data findings are dated,

bounded evidence. The parent owns PDF/Markdown integration and publication. No secrets, screenshots, recordings or production captures were used. No memory files were used by this lane.

Fresh engineering state and completion path

Observed 2026-10-16 PDT / 2026-10-17 UTC. Read-only engineering review; no source implementation, production actions, fresh builds or fresh test suites. Main was independently resolved through GitHub as `ae68aea7febd772282606bf1e142d507316cfea0`; an isolated clean detached clone is `/tmp/tracera-readiness-source-20261016-latest`. GitHub reports fork=false, parent=null. PR #1132 remains open at `723964216ccc6b21b5488a30e501543769808e16` at observation time. Publication may advance that docs branch after this review.

Verdict and evidence boundary

Operational kernel qualified in one isolated Windows/SQLite candidate; valuable product not yet qualified. Q0's source matches current main exactly. Its evidence is stronger than the earlier unavailable-runtime observation and must supersede that observation for the isolated candidate only. The shared service and hosted frontend are not thereby qualified.

The Q0 report attests an explicit-target `cargo build --locked -p tracera-server` (exit 0, 2m36s), archive and binary hashes, readiness 200, invalid query 400, absent record 404, unconfigured empty ingest 422, supplied-issue ingestion, the identical Story after restart, and cleanup of only owned PIDs/listener. No unit suite was executed. The candidate stopped. Reference: `docs/audit/TRACERA-Q0-RUNTIME-QUALIFICATION.md` on PR #1132. These are preserved execution receipts, not a rerun in this review. A persisted Story does not establish canonical product/observation durability, automatic discrepancy discovery, frontend usability, PostgreSQL, or release acceptance.

The empty-input 422 versus stale YAML 200 is a documented contract discrepancy. It does not establish that the handler should return 200. No production IDs or secrets are needed to reproduce that distinction.

Actual progress since the previous readiness review

Main change	Established in source	Remaining boundary
#1149 persisted journeys/steps	Genuine Item reads, complete paged collection, project/type checks, explicit known-empty versus missing steps	Writes/run unavailable; frontend acceptance unproven
#1148 project JSON export/import	Canonical Items and internal artifact trace links round-trip; rejects incompatible shapes; partial writes reported truthfully	Not full product backup; no transaction; no durable retry/idempotency contract
#1147 auth ownership	Legacy login/logout/refresh return typed 501 and name upstream boundary	Upstream deployment protection remains unproven
#1146 SQLite title search	Bounded title substring search and suggestions, project filtering, stable order	No relevance/symbol/semantic index; ASCII case-folding; index APIs unavailable
#1141 specification reads	Persisted type-filtered list/detail/stats, incomplete fields remain null	Summary, verify/run/activities and mutations unfinished

These are useful incremental completions. A typed 501 removes fabricated success but does not deliver the missing capability.

Current engineering goal clusters

Cluster	Source state and useful assets	Acceptance still required
Storage	Store abstractions; SQLite/PostgreSQL adapters and migrations; Q0 SQLite Story restart witness	Semantic observations, baselines, findings, retry identities and provenance survive restart and failure
Project identity	`sqlite_store/projects.rs:17-23,74-80` and PostgreSQL equivalent derive projects from nondeleted problems	First-class or consistently derived identity includes real Item-only products; no false Project Not Found

Cluster	Source state and useful assets	Acceptance still required
Source import	`ingest/persist.rs:39-111` persists Story/evidence/trace links	Namespaced external identity, replay/update semantics, atomicity or explicit resumable partial progress; cross-source collision controls
Product semantics	Product/intent/observation/baseline/assessment types, traversal and detector implementations exist	Mount durable product-aware import and query contracts; link actual source revisions to accepted intent
Observation ingestion	`product/ingest.rs:3-8,70-86` explicitly stores values and idempotency keys in memory	Persistent backend and mounted pipeline; idempotency across restart and partial batches
Detectors	Pure freshness/coverage/contradiction functions with tests	Correct product/capability scoping, comparable observations, persisted findings and execution through an actual user workflow
Typed artifacts	Item/spec/journey projections preserve canonical documents; unsupported metrics are null	Full artifact-X/view-X UI parity, complete pages, correct detail IDs, matching metrics and source evidence
Search	SQLite title substring and suggestions are real	Product-appropriate discovery and bounded evidence navigation; no claim of semantic/symbol search
Snapshots	Items and internal artifact links only, at most 1000 Items	Product models/observations/baselines/evidence semantics; consistent snapshot and atomic/replay restore
Graph/evidence	Graph algorithms, artifact links and evidence records reusable	Explain a discrepancy and affected consumers with revision-qualified anchors and retained uncertainty
Auth/isolation	Optional common bearer gate and upstream auth integrations	Deployment configuration, user/project authorization and cross-product access controls demonstrated
HTTP write/ALM breadth	Many route families explicitly unavailable	Retain or explicitly disposition each promised capability; never silently delete historical obligations
MCP/CLI	Separate Rust MCP node/edge tools and DB selection; explicit demo mode	Identity/contract parity with canonical product, durable failures, tool-driven core workflow, CLI acceptance
Deploy/recovery	Q0 build/readiness/restart/cleanup qualified in disposable candidate	Hosted ingress/frontend, supervised install/update, rollback/recovery, PostgreSQL and release proof
Quality	Useful failure, pagination, type, snapshot and unknown-data tests exist	Execute relevant suites and independent negative controls; enforce numeric coverage and local quality gates

Specific high-priority semantic defects

1. `product/detectors.rs:129-130` sets a freshness finding's capability ID to its product ID, despite `Observation.capability_id` existing (`observation.rs:93`). This conflates identities.

2. `product/detectors.rs:179-193` tests whether any observation's product ID equals a capability ID. A properly modelled observation for product P/capability C can therefore fail coverage association.

3. `product/detectors.rs:237-264` groups contradiction observations by product and baseline, omitting capability and observation kind. Unrelated passing and failing observations can become a false contradiction, again labelling product ID as capability ID. Add counterexamples before using this detector for user decisions.

4. Product observation ingestion and detector suite have no demonstrated mounting or durable Store wiring in current router/main/Store implementations. Tables in migration `009_product_model.sql` provide nodes/edges, not proof observations/findings run durably.

5. `ingest/persist.rs:39` uses `story-{external_id}`, without source/project namespace; separate Story/evidence/link writes can partially succeed. Collision and retry behavior need tests; production corruption was not observed or asserted.

6. `routes/export_import.rs:1-2,152-239` explicitly implements a narrow item snapshot with sequential writes and partial-write reporting. That honesty is valuable but cannot substitute for atomic semantic ingestion or complete backup.

Unavailable surface: reproducible lower bound, not total feature count

Static census: **39 modules contain 216 unique literal path strings in ROUTES arrays directly passed to** `unavailable_routes(ROUTES)`. Method: read each `crates/tracera-server/src/routes/*.rs`, select that exact call, extract literal ROUTES array entries, deduplicate exact strings globally. Excludes mixed modules, dynamic routes, aliases, fallback-only unsupported methods, top-level router registrations and intentionally upstream-owned auth. It is a source-level lower bound of exact templates, not 216 independent business capabilities or a percentage of the product.

Family group	Exact-template count within that census
Item specification operations	43
Integrations	23
Executions + execution configuration	11
Test cases + test runs + test suites	25
Coverage + spec analytics + QA + quality	21
Storage	9
Projects + problems + processes	19
All other selected modules	65
Total deduplicated selected templates	216

Other unavailable contracts include specification overview/stats, verifies, activities, scenario execution (`routes/specs.rs:272-284`), search indexing, canonical link operations and generic router operations. Historical 29 audit findings, 284 API references, 652 baseline records, and view counts describe different denominators. Do not add or divide them into a completion percentage.

Test mocks are legitimate test infrastructure. MCP `TRACERA_DEMO=1` is an explicitly selected in-memory demo path (`crates/tracera-mcp/src/bin/mcp-server.rs:82-87,122-139`); its existence is not evidence that configured DB-backed production is mocked. Conversely, unavailable mounted business operations are real unfinished scope even though their error contracts are tested.

Quality qualification

Tests inspected, not rerun: specification list/detail/stats persistence, exact 501/no-write behavior, wrong-type/project/query/pagination controls and incomplete-metric preservation (`routes/specs_tests.rs:84-281`); journey pagination and malformed-vs-empty steps (`routes/journeys.rs:281-422`); JSON round-trip, invalid input no-write, partial-write disclosure and store failure (`routes/export_import.rs:324-478`). Q0 proves build and narrow HTTP behavior only.

`.github/workflows/coverage.yml:29-39` emits Rust coverage without a numeric fail-under. Frontend `vitest.config.ts:13-24` declares 90% thresholds, but thresholds require coverage mode and cannot establish a

measured result. `lefthook.yml:59,143` masks Ruff-format and Python-collection failure with `|| true`. No current coverage percentage, hosted-green status or broad test acceptance is claimed.

Dependency-ordered completion work

Package	Concrete change/acceptance	Dependency	Effort range (engineer-days)
A: accepted boundary/runtime	Preserve wider baseline; select canonical runtime/identity and acceptance oracles; Q0 already exists	Operator intent/dossier	1-3
B: canonical durable model	Durable products, capabilities, accepted baseline revisions, observations, source anchors, findings; mounted queries/import	A	5-9
C: correct ingestion	Source-qualified IDs; repeat/restart/partial-write semantics; import three owned products without cross-product collisions	B	4-8
D: genuine discrepancy loop	Correct detector identity/comparability; persisted explanation; source change/reassessment; positive and independent negative controls	B,C	6-12
E: usable complete artifact UI	Project identity, typed navigation/detail/pagination, source links and honest summary; actual unfamiliar-user tasks	B-D; partly parallel	5-9
F: recovery/isolation/pilot	Failure injection, restart, access control, supervised deploy and three-product unfamiliar-user pilot	C-E	5-10

Ranges overlap and are not a naive additive schedule. The unchanged semantic blockers dominate. Completing Q0 reduces operational uncertainty but does not justify subtracting all prior qualification days or compressing the semantic implementation.

Cumulative milestone from current source	Optimistic	Base	Conservative
Valuable CVP	14-24 engineer-days	24-44 engineer-days	44-74 engineer-days
Unfamiliar-user pilot	18-30 engineer-days	28-50 engineer-days	50-85 engineer-days
Broad retained v1 including disposition of every obligation	65-110 engineer-days	95-215 engineer-days	180-320+ engineer-days

Planning estimates, low confidence, not measured velocity or commitments. A one-day shift of the old base is intentionally modest: Q0 is real but only part of qualification. Broad v1 is not bounded above until retained scope and dynamic dependencies are agreed. Illustrative two-engineer capacity of 1.2-1.5 effective engineer-days/day makes base CVP roughly 4-8 working weeks, pilot roughly 4-9, broad v1 roughly 13-36; no actual staffing or throughput guarantee.

Rewrite, feedback and sunset

Retain storage/projections/tests and replace narrowly where controls expose failure. A wholesale rewrite does not solve the still-ambiguous retained product boundary and sacrifices usable persistence infrastructure. First ship a complete narrow product loop: import real intent/source, discover a genuine discrepancy, explain source evidence, reconcile source change, restart with the same model. A pile of 501-to-200 route changes is not that loop.

User intent feedback should happen now; usability/value feedback requires the functioning loop. No unfamiliar-user three-product pilot is established. Scaling work follows actual value and measured limits. Sunset only after an explicit bounded pilot/decision, retaining audit/baseline evidence; feedback/intent evolution must explicitly disposition old obligations, never silently erase them.

Memory provenance

A lightweight prior-memory pass used `MEMORY.md:820-822` only to preserve the rule that incomplete tests are not green. Its old adapter run was not reused as current evidence. Rollout ID `019f6924-7ca3-72b3-b28b-ea1ed6829124`. All engineering conclusions above derive from current main and revision-matched Q0 documentation.

Tracera: forward path to a valuable product and retained-scope completion

Planning pin: `ae68aea7febd772282606bf1e142d507316cfea0`, reviewed 2026-10-16 PDT / 2026-10-17 UTC. This is an evidence-qualified plan, not execution or a delivery promise. Q0 operational kernel is qualified in one stopped disposable Windows/SQLite candidate. Product CVP remains NOT READY.

Finish the coherent job first

A valuable CVP/MVP has narrower breadth and complete behavior: import a real owned product, connect accepted intent and actual source/evidence, discover one genuine discrepancy without manually pre-authoring the finding, explain it with correct anchors, reconcile source change, and retain identities/provenance after restart. Requirements, ADRs and tests support this job. A page of 533 records, a green backend readiness route or many truthful 501s does not complete it.

The first real witness can use Tracera itself. Checked-in `tests/e2e/contract/ingest-github.e2e.yml:11-15` predicts 200 for an empty import; handler `handlers/ingest_api.rs:110-122` returns 422 when no live source is configured. Q0 observed the implementation response in isolation. The discrepancy is **stale/inconsistent contract evidence**, not proof that 200 is desired. The product must discover and explain that disagreement without conflating the contract's age with the correct behavior. The distinct-product/capability detector counterexamples are the stronger semantic correctness witness.

```
Q0 operational receipt [DONE, candidate stopped]
                |
                v
A: freeze accepted core + canonical runtime + independent oracles
                |
                v
B: persistent products/capabilities/baselines/observations/findings
                |
                v
C: complete, namespaced, atomic/replay-safe source import
                |
                v
D: correct detectors -> explain discrepancy -> source-change reconcile
                |
                v
E: actual typed frontend, exact artifacts, bounded evidence navigation
                |
                v
F: failure/restart/isolation/deployment + unfamiliar-user pilot
                |
                v
retain-and-finish breadth / revise intent / explicit sunset
```

Pure detector counterexamples and UI inventory can proceed alongside schema planning, but shared schema/Store/router integration has one owner. Avoid a parallel fleet racing on shared traits. No production mutation, merge or rewrite is implied by this report.

Concrete work packages and acceptance

Package	Scope and concrete result	Dependencies	Effort, engineer-days	Primary uncertainty
A. Boundary and artifact qualification	Record retained obligations, canonical runtime/profile and accepted job; freeze source manifest and independent positive/negative oracles; reuse Q0	Owner intent and current receipts	1-3	Product meaning and which profiles must ship

Package	Scope and concrete result	Dependencies	Effort, engineer-days	Primary uncertainty
B. Durable canonical product model	Products/capabilities/accepted baseline revisions, anchors, observations/findings and receipts persist through mounted Store adapters	A	5-9	Schema migration and distinct existing model ownership
C. Correct source import	Source-qualified external IDs; full paging; idempotency across retry/restart; same key/different bytes conflict; atomic batch or explicitly accepted resumable contract	B	4-8	Current separate writes, collision/replay semantics
D. Genuine semantic loop	Fix identity/comparability; real producer observations; explain mismatch, preserve unknowns; source R+1 updates historical/current findings correctly	B,C	6-12	Detector false positives and source-to-model bridge
E. Usable frontend	Item-only projects discoverable; artifact-X/view-X; complete pages/detail; evidence-backed summary and bounded explanation journey	B-D, partly parallel	5-9	Frontend/backend contract and real navigation
F. Reliability and pilot	Cross-product/private-evidence controls; crash/restart/restore; supported ingress/install; independent unfamiliar-user three-product tasks	C-E	5-10	Deployment and unanticipated user failures

Ranges overlap; summing rows mechanically is not a schedule. Each includes focused implementation, meaningful tests, review and evidence. Uncertainty should shrink after A/B. Report estimate changes as old -> new with measured evidence and remaining dependencies.

Time to useful and finished

No measured implementation velocity or committed staffing exists. Estimates below are planning judgement with low confidence, assuming reuse of Rust/React/Store foundations, one backend/integration owner plus one frontend/acceptance owner, actual owned sample products and access to the supported environment. Waiting on users/access is separate elapsed time. Agent concurrency is not linear engineer capacity.

Cumulative milestone from current source	Optimistic	Base	Conservative	Acceptance meaning
Valuable CVP	14-24 engineer-days	24-44 engineer-days	44-74 engineer-days	One real discrepancy loop, selected real frontend and restart/negative controls
Unfamiliar-user pilot	18-30 engineer-days	28-50 engineer-days	50-85 engineer-days	Three owned products, source change/restart, independent operator and value evidence

Cumulative milestone from current source	Optimistic	Base	Conservative	Acceptance meaning
Broad retained v1	65-110 engineer-days	95-215 engineer-days	180-320+ engineer-days	Every explicitly retained obligation has correct implementation and accepted supported-profile evidence

Illustration only: two experienced engineers achieving 1.2-1.5 effective engineer-days per working day imply base CVP roughly 4-8 working weeks, pilot roughly 4-9, broad v1 roughly 13-36. No actual capacity, start date or finish date is asserted. Q0's completion modestly reduces the old 25-45 CVP base to 24-44; it does not eliminate the semantic pipeline. Broad v1 has no defensible upper bound until historical retained scope and dynamic dependencies are decided. Estimates are not a reason to silently retire obligations.

Finishing the whole retained project

After the first coherent job, disposition every baseline record without losing provenance. The seven indexed sources and 652 promise records are discovery material, not an accepted backlog of 652 equally sized tasks. Maintain a closure ledger with source promise, current contract, implementation owner, evidence profile, dependency, disposition and user impact. Suggested sequencing:

Retained family	Finish requirement	Prioritization rationale
Product/identity/import/model	Correct complete durable semantic model and replay/reconciliation	Core dependency for every meaningful explanation
Source/evidence/graph/search	Qualified anchors, comparability, bounded navigation, affected consumers, explicit unsupported edges	Makes findings understandable and trustworthy
Artifact/specification UI	Exact type/product/data completeness, real detail/summary and verification evidence	Prevents misleading visible success
Authoring/work management	Real retained create/edit/link/workflow operations, validation and durable history	Finish promised breadth after model ownership settles
QA/execution/analytics/reporting	Actual run producers, result provenance, correct denominators and non-fabricated reporting	Records alone are not execution or compliance
Integrations/collaboration	Real scoped roundtrips, conflict/retry/privacy controls and failure contracts	Each integration adds external lifecycle cost
Agent traces/sessions/memory	Demonstrated durable contribution to accepted product purpose	Resolve observability vs product-model intent explicitly
UI/API/MCP/CLI profiles	Same canonical IDs, ownership, errors and outcome across required interfaces	Avoid separate demos masquerading as parity
Security/release/operations	Accepted authorization, backup/recovery, install/update and supported artifact matrix	Local build differs from released and installed acceptance
Documentation/quality/closure	Truthful capabilities, enforced numeric gates, independent negative controls, dated closure receipts	Completion is evidence, not route or task counts

Do not spend the whole broad-v1 allowance converting stubs individually without grouping capabilities by user outcome. One route can support several jobs; several aliases can represent one capability. Explicit unavailability remains preferable to fabricated successful mocks while implementing real behavior.

Rewrite decision

Recommend retain and incrementally complete, with targeted replacement where qualified controls fail. Existing storage adapters, migrations, typed projections, graph algorithms, persistence tests and honest error behavior are useful

assets. Wholesale rewrite would re-create them while leaving product intent and acceptance unresolved.

Before replacing a subsystem, write a bounded retain/adopt/patch/wrap comparison: real consumer/job, current failure evidence, established local or external alternative, preserved identity/data/privacy, migration/rollback, maintenance burden and independent proof. Native indexing/search and native test/report formats are reuse directions in the dossier, not approval to invent a universal new engine. An adapter or narrower promise can be a successful product decision when explicitly accepted. Do not publish into other people's upstream repositories.

Feedback, scaling, intent evolution and sunset

- **Feedback now:** validate intended question, authoritative evidence and retained scope. This is intent feedback, not product-fit acceptance.

- **Feedback after CVP:** independent user attempts, not a developer-led demo. First one-product protocol, then three-product pilot with seeded and retained-real gaps. Record failures/interventions and compare with competent manual workflow.

- **Scaling:** defer speculative throughput architecture until useful behavior is established and measured limits are real. Correct identity, privacy, durability and bounded behavior are immediate correctness requirements, not optional future scaling.

- **Intent evolution:** document accepted changes and migration for existing consumers; preserve superseded promises/evidence. ALM or observability cannot silently replace the governed product-model job.

- **Sunset:** a bounded pilot shows no useful advantage, an owned tool already fulfils the accepted job more simply, or operator explicitly chooses another direction. Retain/export necessary data and evidence, state which obligations move to which owner, stop owned services safely and leave a durable handoff. Sunset is an explicit product decision, not a convenient label for unfinished code.

Immediate next bounded ownership

One owner should freeze the durable product-observation/import contract and mount the smallest real pipeline; an independent reviewer owns distinct-product/capability and contradiction controls. The next meaningful proof is not another static audit: imported real source expectation + independent observation -> persisted finding -> real UI explanation -> source change -> restart, without hand-authoring the finding. First execution slice and PR scope must remain isolated from existing checkouts and production, with no merge or force push.

No memory files were used by this lane. Current engineering details and exact source anchors are in companion `02-ENGINEERING-STATE.md`; governing authority is `DOSSIER.md:5-24,55-93` and `AGENT-PROMPT.md:15-25,37-45` in the external docs-5 Tracera dossier.